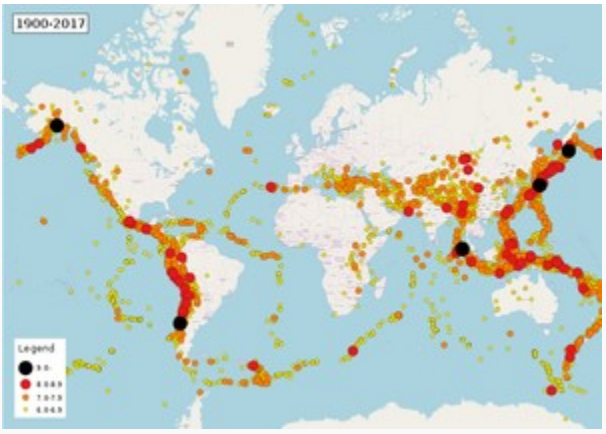
Qt for Python Tutorial: Data Visualization Tool

Motivation



Earthquakes (M6.0+) between 1900 and 2017 (Source: Wikipedia)

There are many sources of **open data** that one can use for interesting project, from statistics on social networks, to information from sensors all over the world.

One of the examples for this is the U.S. Geological Survey which provides updated information regarding the earthquakes we have in the last hours, day, week, and month (You can visit the website and download the CSV files with this information.

Even though they provide filtered information related to the **magnitude of the earthquakes**, we will try to use the raw data (*all_day.csv*, *all_hour.csv*, ...) to deal with missing information, or even incorrect data, so we can filter the data first just for the example.

Resources

Some useful resources to understand the details of this tutorial are:

- Qt Model View Programming
- QMainWindow details diagram
- QAbstractTableModel subclassing requirements
- QtCharts examples
- Python Pandas tutorial
- Python argparse tutorial

Tutorial

First step: Command line options and reading the data

There are many ways of reading data from Python, and this will not recommend the optimal way of doing it, but just state one of the alternatives out there.

For this example you can also try:

- Native file reading,
- The csv module
- The numpy module
- among others.

We will use pandas, because it provides a simple way of reading and filtering data.

Additionally, we can pass the data file we want to use via command line, and for this one can use the built-in sys module to access the argument of the script but luckily there are better ways to achieve this, like the argparse module. Using argparse allow us to have a simple interaction command line interface for our project, so let's take a look how a first attempt will look like:

```
 1 import argparse
 2 import pandas as pd
 3
 4 def read_data(fname):
 5     return pd.read_csv(fname)
 6
 7 if __name__ == "__main__":
 8     options = argparse.ArgumentParser()
 9     options.add_argument("-f", "--file", type=str, required=True)
10     args = options.parse_args()
11     data = read_data(args.file)
12     print(data)
```

This will allow us to execute our script and use the option **-f/--file** to point to the data file we want to use. So far we only read the whole CSV file, and we will need more than using the **read_csv** function to get our data properly.

Let's look what happens when we execute this code:

```
$ python first_step.py -f all_hour.csv
                         time   latitude   longitude  depth   mag magType
...    magError magNst     status  locationSource  magSource
0   2018-12-11T21:14:44.682Z  61.384300 -150.124800  42.00  1.60      ml
...         NaN    NaN  automatic              ak         ak
```

```
1   2018-12-11T21:12:26.250Z  33.039333 -115.594000   5.33  1.94      ml
...       0.171   26.0  automatic              ci         ci
2   2018-12-11T21:07:02.435Z  61.465600 -149.980800  34.50  1.50      ml
...          NaN    NaN  automatic              ak         ak
3   2018-12-11T21:04:17.794Z  61.397600 -150.087900  39.80  1.60      ml
...          NaN    NaN  automatic              ak         ak
4   2018-12-11T21:01:27.480Z  33.222167 -115.565833  13.03  1.51      ml
...       0.257   17.0  automatic              ci         ci
5   2018-12-11T20:58:11.448Z  61.393400 -150.092900  39.60  1.40      ml
...          NaN    NaN  automatic              ak         ak
6   2018-12-11T20:56:17.290Z  33.498167 -116.803833   2.71  0.43      ml
...       0.097    8.0  automatic              ci         ci
7   2018-12-11T20:55:14.585Z  61.463500 -149.963100  33.10  1.60      ml
...          NaN    NaN  automatic              ak         ak
8   2018-12-11T20:48:27.290Z  33.494333 -116.801000   2.28  1.89      ml
...       0.191   26.0  automatic              ci         ci
9   2018-12-11T20:46:40.780Z  34.638500 -117.113000  -1.23  1.53      ml
...       0.108   16.0   reviewed              ci         ci
10  2018-12-11T20:46:29.115Z  61.440000 -149.993700  53.50  1.70      ml
...          NaN    NaN  automatic              ak         ak
11  2018-12-11T20:44:20.822Z  61.379300 -150.086500  38.20  1.40      ml
...          NaN    NaN  automatic              ak         ak
12  2018-12-11T20:43:38.050Z  33.035500 -115.590500   6.10  1.95      ml
...       0.167   27.0  automatic              ci         ci
13  2018-12-11T20:40:01.109Z  61.487900 -149.908300  36.20  1.60      ml
...          NaN    NaN  automatic              ak         ak
14  2018-12-11T20:34:43.471Z  61.474800 -150.024200  40.50  1.60      ml
...          NaN    NaN  automatic              ak         ak

[15 rows x 22 columns]
```

Note: *The output will differ depending on how wide is your screen and which all_hour.csv file you got.*

Second step: Filtering data and Timezones

As you noticed, in the previous step the data was still on a raw state, so now the idea is to select which columns do we need and how to properly handle it.

For this example we will care of only two columns: **Time** (time) and **Magnitude** (mag). After getting the information of these columns, we will need to filter and adapt the data. Since we want to include the date on a Qt application we will try to format it to Qt types.

For the **Magnitude** we don't have much to do since it's just a floating point number, but we need to take special care if the data is correct. This could be done by filtering the data that follows the condition "**the magnitude must be greater than zero"**, since of course devices could report faulty data, or unexpected behavior.

For the **Date** we already saw it was on a *UTC format* (e.g.: 2018-12-11T21:14:44.682Z), so we could easily map it to a **QDateTime** object defining the structure of the string. Additionally, we can adapt the time to the timezone we are living, so we get some sense of the time. This could be done by a **QTimeZone**.

Let's take a look of how the code will look like:

```
import argparse
import pandas as pd

from PySide2.QtCore import QDateTime, QTimeZone

def transform_date(utc, timezone=None):
    utc_fmt = "yyyy-MM-ddTHH:mm:ss.zzzZ"
    new_date = QDateTime().fromString(utc, utc_fmt)
    if timezone:
        new_date.setTimeZone(timezone)
    return new_date

def read_data(fname):
    # Read the CSV content
    df = pd.read_csv(fname)

    # Remove wrong magnitudes
    df = df.drop(df[df.mag < 0].index)
    magnitudes = df["mag"]

    # My local timezone
    timezone = QTimeZone(b"Europe/Berlin")

    # Get timestamp transformed to our timezone
    times = df["time"].apply(lambda x: transform_date(x, timezone))

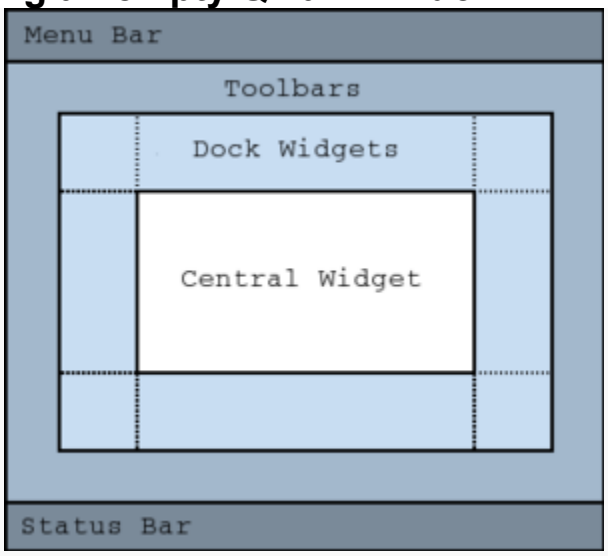
    return times, magnitudes


if __name__ == "__main__":
    options = argparse.ArgumentParser()
```

```
32      options.add_argument("-f", "--file", type=str, required=True)
33      args = options.parse_args()
34      data = read_data(args.file)
35      print(data)
```

Now we will have a tuple of **QDateTime** and **float** data that we can use for the next steps.

Third step: Creating an empty QMainWindow



QMainWindow layout

Let's start adding a "face" to our code, and for this example we will use a QMainWindow. The idea of using it is that the structure we are getting is quite convenient for this kind of applications, take a look at the diagram on the right.

While using QMainWindow we get a Menu Bar and a Status Bar *for free* so it is easier to add element to them.

- We will add a Menu called "File" and include a QAction called "Exit" to close the window.
- For the status bar we can show a message once the application starts.
- The size of the window can be fixed by hand or you can adjust it related to the resolution you currently have. Inside the following snippet you can find a solution to use the available 80% of the width and 70% of the height in your screen.

Keep in mind that since we have a QMainWindow we need to modify our main section to include the usual code regarding creating a QApplication instance and showing the QMainWindow.

Note: You can still achieve a similar structure using other Qt elements like QMenuBar, QWidget and QStatusBar, but you will need to take care of the MainWindow layout and design, while QMainWindow already has a layout structure (Right diagram).

```
 1 import sys
 2 import argparse
 3 import pandas as pd
 4
 5 from PySide2.QtCore import (QAbstractTableModel, QDateTime, QModelIndex,
 6                             QRect, Qt, QTimeZone, Slot)
 7 from PySide2.QtGui import QColor, QPainter
 8 from PySide2.QtWidgets import (QAction, QApplication, QHBoxLayout,
QHeaderView,
 9                                QMainWindow, QSizePolicy, QTableView,
QWidget)
10 from PySide2.QtCharts import QtCharts
11
12
13 def transform_date(utc, timezone=None):
14     # ...
15
16 def read_data(fname):
17     # ...
18
19
20 class MainWindow(QMainWindow):
21     def __init__(self):
22         QMainWindow.__init__(self)
23         self.setWindowTitle("Eartquakes information")
24
25         # Menu
26         self.menu = self.menuBar()
27         self.file_menu = self.menu.addMenu("File")
28
29         ## Exit QAction
30         exit_action = QAction("Exit", self)
31         exit_action.setShortcut("Ctrl+Q")
32         exit_action.triggered.connect(self.exit_app)
33
34         self.file_menu.addAction(exit_action)
35
36         # Status Bar
37         self.status = self.statusBar()
38         self.status.showMessage("Data loaded and plotted")
```

```
39
40          # Window dimensions
41          geometry = app.desktop().availableGeometry(self)
42          self.setFixedSize(geometry.width() * 0.8, geometry.height() * 0.7)
43
44      @Slot()
45      def exit_app(self, checked):
46          sys.exit()
47
48
49  if __name__ == "__main__":
50      options = argparse.ArgumentParser()
51      options.add_argument("-f", "--file", type=str, required=True)
52      args = options.parse_args()
53      data = read_data(args.file)
54
55      # Qt Application
56      app = QApplication(sys.argv)
57
58      window = MainWindow()
59      window.show()
60
61      sys.exit(app.exec_())
```

Fourth step: Adding a QTableView to display the data

Now that we have a QMainWindow we can include a **centralWidget** to our interface, and for this we will use a **QWidget** to display our data inside.

The first an easiest way of presenting data is a Table, so we will able to display the content of the file in our application.

The first approach will be to horizontal layout with just a **QTableView**, for this we can just create a QTableView object and place it inside a **QHBoxLayout**. Once the QWidget is properly built we will pass the object to the QMainWindow for it to place it as a central widget.

Remember that a QTableView **needs** a model to display information. In our case we will use a **QAbstractTableModel**.

If you want to use a default item model you could use a **QTableWidget** instead, but in this example we wanted to modify how we were displaying the items in our table.

Implementing the model for our QTableView will allow us to set the headers, manipulate the formats of the cell values (remember we have UTC time and float numbers!), setting

style properties like text alignment, and even setting color properties for the cell or its content.

Subclassing QAbstractTable require us to implement the methods **rowCount()**, **columnCount()** and **data()**, so take special care of handling them properly. Additionally to those methods, we are including **headerData()** just to show the header information.

The process is really simple, and once you have everything in place you can connect the Table with the model doing something like:

```
# Getting the Model
model = CustomTableModel(data)

# Creating a QTableView
table_view = QTableView()
table_view.setModel(model)
```

Of course we need to actually write the CustomTableModel and some other details for the table, but you can take a look at the code for this step:

```
import sys
import argparse
import pandas as pd

from PySide2.QtCore import (QAbstractTableModel, QDateTime, QModelIndex,
                            Qt, QTimeZone, Slot)
from PySide2.QtGui import QColor
from PySide2.QtWidgets import (QAction, QApplication, QHBoxLayout,
QHeaderView,
                               QMainWindow, QSizePolicy, QTableView,
QWidget)


class CustomTableModel(QAbstractTableModel):
    def __init__(self, data=None):
        QAbstractTableModel.__init__(self)
        self.load_data(data)

    def load_data(self, data):
        self.input_dates = data[0].values
        self.input_magnitudes = data[1].values

```

```
        self.column_count = 2
        self.row_count = len(self.input_magnitudes)

    def rowCount(self, parent=QModelIndex()):
        return self.row_count

    def columnCount(self, parent=QModelIndex()):
        return self.column_count

    def headerData(self, section, orientation, role):
        if role != Qt.DisplayRole:
            return None
        if orientation == Qt.Horizontal:
            return ("Date", "Magnitude")[section]
        else:
            return "{}".format(section)

    def data(self, index, role = Qt.DisplayRole):
        column = index.column()
        row = index.row()

        if role == Qt.DisplayRole:
            if column == 0:
                raw_date = self.input_dates[row]
                date = "{}".format(raw_date.toPython())
                return date[:-3]
            elif column == 1:
                return "{:.2f}".format(self.input_magnitudes[row])
        elif role == Qt.BackgroundRole:
            return QColor(Qt.white)
        elif role == Qt.TextAlignmentRole:
            return Qt.AlignRight

        return None

class Widget(QWidget):
    def __init__(self, data):
        QWidget.__init__(self)

        # Getting the Model
        self.model = CustomTableModel(data)
```

```
 62
 63         # Creating a QTableView
 64         self.table_view = QTableView()
 65         self.table_view.setModel(self.model)
 66
 67         # QTableView Headers
 68         self.horizontal_header = self.table_view.horizontalHeader()
 69         self.vertical_header = self.table_view.verticalHeader()
 70
self.horizontal_header.setSectionResizeMode(QHeaderView.ResizeToContents)
 71
self.vertical_header.setSectionResizeMode(QHeaderView.ResizeToContents)
 72         self.horizontal_header.setStretchLastSection(True)
 73
 74         # QWidget Layout
 75         self.main_layout = QHBoxLayout()
 76         size = QSizePolicy(QSizePolicy.Preferred, QSizePolicy.Preferred)
 77
 78         ## Left layout
 79         size.setHorizontalStretch(1)
 80         self.table_view.setSizePolicy(size)
 81         self.main_layout.addWidget(self.table_view)
 82
 83         # Set the layout to the QWidget
 84         self.setLayout(self.main_layout)
 85
 86 def transform_date(utc, timezone=None):
 87     # ...
 88
 89
 90 def read_data(fname):
 91     # ...
 92
 93
 94 class MainWindow(QMainWindow):
 95     def __init__(self, widget):
 96         # ...
 97         self.setCentralWidget(widget)
 98
 99     @Slot()
100     def exit_app(self, checked):
```

```
101         sys.exit()
102
103
104 if __name__ == "__main__":
105     options = argparse.ArgumentParser()
106     options.add_argument("-f", "--file", type=str, required=True)
107     args = options.parse_args()
108     data = read_data(args.file)
109
110     # Qt Application
111     app = QApplication(sys.argv)
112
113     # QWidget
114     widget = Widget(data)
115     # QMainWindow using QWidget as central widget
116     window = MainWindow(widget)
117
118     window.show()
119     sys.exit(app.exec_())
```

Fifth step: Adding a QChartView

A table is not enough, and of course we would like to plot our data. For this, you have the **QtCharts** module which provide many types of plots and options to graphically represent data.

The placeholder for a plot is a **QChartView**, and inside that Widget we can place a **QChart**. As a first step we will include this but without any information.

Take a look of the following code inside the Widget class. This includes the **QChartView** inside the **QHBoxLayout** to position it on the right of the table.

```
 1 import sys
 2 import argparse
 3 import pandas as pd
 4
 5 from PySide2.QtCore import (QAbstractTableModel, QDateTime, QModelIndex,
 6                             QRect, Qt, QTimeZone, Slot)
 7 from PySide2.QtGui import QColor, QPainter
 8 from PySide2.QtWidgets import (QAction, QApplication, QHBoxLayout,
QHeaderView,
 9                                QMainWindow, QSizePolicy, QTableView,
QWidget)
```

```
from PySide2.QtCharts import QtCharts


class CustomTableModel(QAbstractTableModel):
    # ...

class Widget(QWidget):
    def __init__(self, data):
        QWidget.__init__(self)

        # Getting the Model
        self.model = CustomTableModel(data)

        # Creating a QTableView
        self.table_view = QTableView()
        self.table_view.setModel(self.model)

        # QTableView Headers
        self.horizontal_header = self.table_view.horizontalHeader()
        self.vertical_header = self.table_view.verticalHeader()

self.horizontal_header.setSectionResizeMode(QHeaderView.ResizeToContents)

self.vertical_header.setSectionResizeMode(QHeaderView.ResizeToContents)
        self.horizontal_header.setStretchLastSection(True)

        # Creating QChart
        self.chart = QtCharts.QChart()
        self.chart.setAnimationOptions(QtCharts.QChart.AllAnimations)

        # Creating QChartView
        self.chart_view = QtCharts.QChartView(self.chart)
        self.chart_view.setRenderHint(QPainter.Antialiasing)

        # QWidget Layout
        self.main_layout = QHBoxLayout()
        size = QSizePolicy(QSizePolicy.Preferred, QSizePolicy.Preferred)

        ## Left layout
        size.setHorizontalStretch(1)
        self.table_view.setSizePolicy(size)
```

```
        self.main_layout.addWidget(self.table_view)

        ## Right Layout
        size.setHorizontalStretch(4)
        self.chart_view.setSizePolicy(size)
        self.main_layout.addWidget(self.chart_view)

        # Set the layout to the QWidget
        self.setLayout(self.main_layout)


def transform_date(utc, timezone=None):
    # ...


def read_data(fname):
    # ...


class MainWindow(QMainWindow):
    # ...

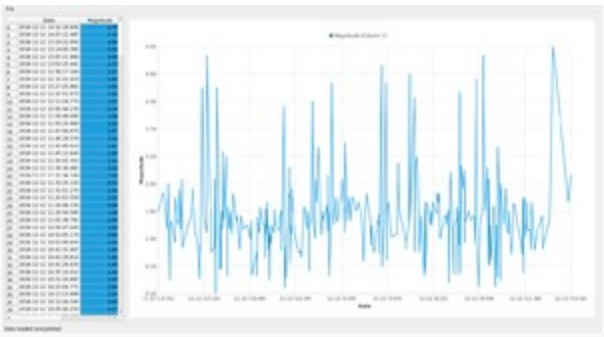

if __name__ == "__main__":
    options = argparse.ArgumentParser()
    options.add_argument("-f", "--file", type=str, required=True)
    args = options.parse_args()
    data = read_data(args.file)

    # Qt Application
    app = QApplication(sys.argv)

    # QWidget
    widget = Widget(data)
    # QMainWindow using QWidget as central widget
    window = MainWindow(widget)

    window.show()
    sys.exit(app.exec_())
```

Final Result



Your first Data Visualization Tool with Qt for Python

The last step of this tutorial is just to include the data inside our **QChart**. For this we just need to go over our data and include the data on a **QLineSeries**.

After adding the data to the series, you can modify the axis to properly display the **QDateTime** on the X-axis, and the magnitude values on the Y-axis.

We also obtain the color of the plot so we can highlight the table column of the plotted data.

Here is the final version of our code, remember to download any data and try it out!

```
  1 import sys
  2 import argparse
  3 import pandas as pd
  4
  5 from PySide2.QtCore import (QAbstractTableModel, QDateTime, QModelIndex,
  6                             Qt, QTimeZone, Slot)
  7 from PySide2.QtGui import QColor, QPainter
  8 from PySide2.QtWidgets import (QAction, QApplication, QHBoxLayout,
QHeaderView,
  9                                QMainWindow, QSizePolicy, QTableView,
QWidget)
 10 from PySide2.QtCharts import QtCharts
 11
 12
 13 class CustomTableModel(QAbstractTableModel):
 14     def __init__(self, data=None):
 15         QAbstractTableModel.__init__(self)
 16         self.color = None
 17         self.load_data(data)
 18
 19     def load_data(self, data):
 20         self.input_dates = data[0].values
```

```
        self.input_magnitudes = data[1].values

        self.column_count = 2
        self.row_count = len(self.input_magnitudes)

    def rowCount(self, parent=QModelIndex()):
        return self.row_count

    def columnCount(self, parent=QModelIndex()):
        return self.column_count

    def headerData(self, section, orientation, role):
        if role != Qt.DisplayRole:
            return None
        if orientation == Qt.Horizontal:
            return ("Date", "Magnitude")[section]
        else:
            return "{}".format(section)

    def data(self, index, role=Qt.DisplayRole):
        column = index.column()
        row = index.row()

        if role == Qt.DisplayRole:
            if column == 0:
                raw_date = self.input_dates[row]
                date = "{}".format(raw_date.toPython())
                return date[:-3]
            elif column == 1:
                return "{:.2f}".format(self.input_magnitudes[row])
        elif role == Qt.BackgroundRole:
            return (QColor(Qt.white), QColor(self.color))[column]
        elif role == Qt.TextAlignmentRole:
            return Qt.AlignRight

        return None


class Widget(QWidget):
    def __init__(self, data):
        QWidget.__init__(self)
```

```
62
63          # Getting the Model
64          self.model = CustomTableModel(data)
65
66          # Creating a QTableView
67          self.table_view = QTableView()
68          self.table_view.setModel(self.model)
69
70          # QTableView Headers
71          resize = QHeaderView.ResizeToContents
72          self.horizontal_header = self.table_view.horizontalHeader()
73          self.vertical_header = self.table_view.verticalHeader()
74          self.horizontal_header.setSectionResizeMode(resize)
75          self.vertical_header.setSectionResizeMode(resize)
76          self.horizontal_header.setStretchLastSection(True)
77
78          # Creating QChart
79          self.chart = QtCharts.QChart()
80          self.chart.setAnimationOptions(QtCharts.QChart.AllAnimations)
81          self.add_series("Magnitude (Column 1)", [0, 1])
82
83          # Creating QChartView
84          self.chart_view = QtCharts.QChartView(self.chart)
85          self.chart_view.setRenderHint(QPainter.Antialiasing)
86
87          # QWidget Layout
88          self.main_layout = QHBoxLayout()
89          size = QSizePolicy(QSizePolicy.Preferred, QSizePolicy.Preferred)
90
91          # Left layout
92          size.setHorizontalStretch(1)
93          self.table_view.setSizePolicy(size)
94          self.main_layout.addWidget(self.table_view)
95
96          # Right Layout
97          size.setHorizontalStretch(4)
98          self.chart_view.setSizePolicy(size)
99          self.main_layout.addWidget(self.chart_view)
100
101         # Set the layout to the QWidget
102         self.setLayout(self.main_layout)
```

```
103
104     def add_series(self, name, columns):
105         # Create QLineSeries
106         self.series = QtCharts.QLineSeries()
107         self.series.setName(name)
108
109         # Filling QLineSeries
110         for i in range(self.model.rowCount()):
111             # Getting the data
112             t = self.model.index(i, 0).data()
113             date_fmt = "yyyy-MM-dd HH:mm:ss.zzz"
114
115             x = QDateTime().fromString(t, date_fmt).toMSecsSinceEpoch()
116             y = float(self.model.index(i, 1).data())
117
118             if x > 0 and y > 0:
119                 self.series.append(x, y)
120
121         self.chart.addSeries(self.series)
122
123         # Setting X-axis
124         self.axis_x = QtCharts.QDateTimeAxis()
125         self.axis_x.setTickCount(10)
126         self.axis_x.setFormat("dd.MM (h:mm)")
127         self.axis_x.setTitleText("Date")
128         self.chart.addAxis(self.axis_x, Qt.AlignBottom)
129         self.series.attachAxis(self.axis_x)
130         # Setting Y-axis
131         self.axis_y = QtCharts.QValueAxis()
132         self.axis_y.setTickCount(10)
133         self.axis_y.setLabelFormat("%.2f")
134         self.axis_y.setTitleText("Magnitude")
135         self.chart.addAxis(self.axis_y, Qt.AlignLeft)
136         self.series.attachAxis(self.axis_y)
137
138         # Getting the color from the QChart to use it on the QTableView
139         self.model.color = "{}".format(self.series.pen().color().name())
140
141
142 def transform_date(utc, timezone=None):
143     utc_fmt = "yyyy-MM-ddTHH:mm:ss.zzzZ"
```

```
144     new_date = QDateTime().fromString(utc, utc_fmt)
145     if timezone:
146         new_date.setTimeZone(timezone)
147     return new_date
148 
149 
150 def read_data(fname):
151     # Read the CSV content
152     df = pd.read_csv(fname)
153 
154     # Remove wrong magnitudes
155     df = df.drop(df[df.mag < 0].index)
156     magnitudes = df["mag"]
157 
158     # My local timezone
159     timezone = QTimeZone(b"Europe/Berlin")
160 
161     # Get timestamp transformed to our timezone
162     times = df["time"].apply(lambda x: transform_date(x, timezone))
163 
164     return times, magnitudes
165 
166 
167 class MainWindow(QMainWindow):
168     def __init__(self, widget):
169         QMainWindow.__init__(self)
170         self.setWindowTitle("Eartquakes information")
171 
172         # Menu
173         self.menu = self.menuBar()
174         self.file_menu = self.menu.addMenu("File")
175 
176         # Exit QAction
177         exit_action = QAction("Exit", self)
178         exit_action.setShortcut("Ctrl+Q")
179         exit_action.triggered.connect(self.exit_app)
180 
181         self.file_menu.addAction(exit_action)
182 
183         # Status Bar
184         self.status = self.statusBar()
```

```
185         self.status.showMessage("Data loaded and plotted")
186
187         # Window dimensions
188         geometry = app.desktop().availableGeometry(self)
189         self.setFixedSize(geometry.width() * 0.8, geometry.height() *
0.7)
190         self.setCentralWidget(widget)
191
192     @Slot()
193     def exit_app(self, checked):
194         sys.exit()
195
196
197 if __name__ == "__main__":
198     options = argparse.ArgumentParser()
199     options.add_argument("-f", "--file", type=str, required=True)
200     args = options.parse_args()
201     data = read_data(args.file)
202
203     # Qt Application
204     app = QApplication(sys.argv)
205
206     # QWidget
207     widget = Widget(data)
208     # QMainWindow using QWidget as central widget
209     window = MainWindow(widget)
210
211     window.show()
212     sys.exit(app.exec_())
```